

Predicción de Precios de Criptomonedas

Análisis con Regresión Lineal Múltiple

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor # Para comparar
from sklearn.metrics import mean_squared_error, r2_score, mean_absolute_error
```

In [2]:

```
#importacion de los datos
data=pd.read_csv('currentPriceCrypto.csv')
data.head()
```

In [3]:

```
data.tail()
```

In [4]:

```
#info de los datos
data.info()
```

In [5]:

```
#describe de los datos
data.describe()
```

In [6]:

```
##revisar si hay valores nulos
data.isnull().sum()
```

In [7]:

```
##grafica para ver la distribucion de precios

import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.hist(data["current_price_usd"], bins=50)
plt.title("Distribución de current_price_usd")
plt.xlabel("Precio")
plt.ylabel("Frecuencia")
plt.savefig('distribucion_current_price_usd',dpi=200)
plt.show()
```

se puede ver que la distribucion esta altamente sesgada a la derecha

In [8]:

```
##grafica para ver el sesgo de los datos

import seaborn as sns
```

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,4))
sns.boxplot(x=data["current_price_usd"])
plt.title("Boxplot de current_price_usd")
plt.savefig('boxplot_current_price_usd',dpi=200)
plt.show()
```

In [9]:

```
##definir una nueva varaibla predictora la cual es el logaritmo de current_price_usd para un
##mejor analisis,,, esto se hace porque current_price_usd presentaba valores demasiado distantes
```

```
import numpy as np

data["log_price"] = np.log1p(data["current_price_usd"])
```

In [10]:

```
##grafica para ver la nueva distribucion de precios

import matplotlib.pyplot as plt

plt.figure(figsize=(8,5))
plt.hist(data["log_price"], bins=50)
plt.title("Distribución de log_price")
plt.savefig('distribucionLogPrice_current_price_usd',dpi=200)
plt.show()
```

se elimino la cola a la derecha y los valores estan mas comprimidos

In [11]:

```
print("Skew original:", data["current_price_usd"].skew())
print("Skew log:", data["log_price"].skew())
```

se redujo la simetria en la variable objetivo

In [12]:

```
#tabla que muestra la correlacion entre las varaibles predicotras
corr = data.select_dtypes(include=['number']).corr()

corr["log_price"].sort_values(ascending=False)
```

In [13]:

```
# Matriz de correlación de variables predictoras
corr1= data.corr(numeric_only=True)

# Gráfico
plt.figure(figsize=(14,10))
sns.heatmap(
    corr1,
    annot=True,      # muestra los valores
    cmap='coolwarm', # colores
    fmt='.2f'
)

plt.title('Matriz de Correlación')
plt.savefig('correlation_current_price_usd',dpi=200)
```

```
plt.show()
```

In [14]:

```
## diagrama de dispersion

import numpy as np

plt.figure(figsize=(8,5))
plt.scatter(
    np.log1p(data["market_cap_usd"]),
    data["log_price"]
)

plt.xlabel("log_market_cap")
plt.ylabel("log_price")
plt.savefig('dispercionLogmarket_current_price_usd', dpi=200)
plt.show()
```

In [15]:

```
## tabla de estadísticas agrupadas

data.groupby("cryptocurrency")["current_price_usd"].agg(
    ["count", "mean", "median", "min", "max"]
).sort_values("mean")
```

In [16]:

```
## modificar al variable cryptocurrency con codificación One-Hot Encoding sobre la variable categórica cryptocurrency para
## realizar un mejor analisis, esto genera un columna para cada tipo de criptomoneda
```

```
data_model = pd.get_dummies(
    data,
    columns=["cryptocurrency"],
    drop_first=True,
    dtype=int
)
```

In [17]:

```
data_model.head()
```

In [18]:

```
## definir variable objetivo
y = data_model["log_price"]
```

In [19]:

```
## definir las variables predictoras, se eliminan variables como fecha que no se utilizan en esta caso
```

```
X = data_model.drop(
    columns=[
        "timestamp",
        "current_price_usd",
        "log_price"
    ]
)
```

In [20]:

```
##verificar la longitud de ambas variables
```

```
print(X.shape)
```

```
print(y.shape)
```

In [21]:

```
print(X.columns.tolist())
```

In [22]:

```
## distribucion de marcket cap por cirptomonedas
plt.figure(figsize=(12,6))

sns.boxplot(
    data=data,
    x="cryptocurrency",
    y="market_cap_usd"
)

plt.xticks(rotation=45)
plt.savefig('distribucionmarketcap_current_price_usd',dpi=200)
plt.show()
```

In [23]:

```
### entrenamiento y prueba
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=2026
)
```

In [24]:

```
### escalar variables

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

In [25]:

```
### entrenar regresion multiple

from sklearn.linear_model import LinearRegression

modelo = LinearRegression()

modelo.fit(
    X_train_scaled,
    y_train
)
```

In [26]:

```
##realizar prediccion
y_pred = modelo.predict(X_test_scaled)
```

In [27]:

```
##Se calculan tres métricas MAE, RMSE Y R^2
```

MAE (Mean Absolute Error)
RMSE (Root Mean Squared Error)
R² (Coeficiente de Determinación)

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae = mean_absolute_error(
    y_test,
    y_pred
)

rmse = mean_squared_error(
    y_test,
    y_pred
) ** 0.5

r2 = r2_score(
    y_test,
    y_pred
)

print("MAE =", mae)
print("RMSE =", rmse)
print("R2 =", r2)
```

In [28]:

```
###Gráfico de valores reales vs. valores predichos

import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(
    y_test,
    y_pred,
    alpha=0.6
)

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    'r--'
)

plt.xlabel("Valor real")
plt.ylabel("Valor predicho")
plt.title("Valores reales vs predichos")
plt.savefig('valoresReales_current_price_usd',dpi=200)
plt.show()
```

In [29]:

```
## evaluar coeficientes

coeficientes = pd.DataFrame({
    "Variable": X.columns,
    "Coeficiente": modelo.coef_
```

```

}))

coeficientes["Abs"] = abs(coeficientes["Coeficiente"])

coeficientes.sort_values(
    "Abs",
    ascending=False
)

```

La regresión múltiple indica que la variable con mayor capacidad explicativa sobre el precio de las criptomonedas es el tipo de criptomoneda (cryptocurrency), representado mediante variables dummy.

Las categorías asociadas a Bitcoin, Ethereum y Solana presentan los coeficientes más altos, evidenciando que pertenecer a estas criptomonedas incrementa significativamente el valor esperado del precio respecto a la categoría de referencia (Algorand).

Por otro lado, las variables de sentimiento social, sentimiento de noticias, volumen de negociación, RSI, volatilidad y Fear & Greed Index muestran coeficientes cercanos a cero, indicando una contribución marginal dentro del modelo.

In [30]:

```

### validacion cruzada
from sklearn.model_selection import cross_val_score

scores = cross_val_score(
    modelo,
    X_train_scaled,
    y_train,
    cv=10,
    scoring="r2"
)

print(scores)
print("Promedio:", scores.mean())

```

In [31]:

```

nueva_obs = X_test.iloc[[0]]

pred_log = modelo.predict(
    scaler.transform(nueva_obs)
)

print(pred_log)

```

In [32]:

```

import numpy as np

pred_precio = np.expm1(pred_log)

print("Precio predicho:", pred_precio[0])

```

In [33]:

```

real = np.expm1(y_test.iloc[0])

print("Real:", real)
print("Predicho:", pred_precio[0])

```

In [34]:

```
nuevo = pd.DataFrame({
    'price_change_24h_percent':[2.5],
    'trading_volume_24h':[12000000],
    'market_cap_usd':[9.5e14],
    'social_sentiment_score':[0.6],
    'news_sentiment_score':[0.5],
    'news_impact_score':[4.2],
    'social_mentions_count':[15000],
    'fear_greed_index':[70],
    'volatility_index':[60],
    'rsi_technical_indicator':[55],
    'prediction_confidence':[80],

    'cryptocurrency_Avalanche':[0],
    'cryptocurrency_Bitcoin':[1],
    'cryptocurrency_Cardano':[0],
    'cryptocurrency_Chainlink':[0],
    'cryptocurrency_Cosmos':[0],
    'cryptocurrency_Ethereum':[0],
    'cryptocurrency_Polkadot':[0],
    'cryptocurrency_Polygon':[0],
    'cryptocurrency_Solana':[0]
})
```

In [35]:

```
nuevo_scaled = scaler.transform(nuevo)

pred_log = modelo.predict(nuevo_scaled)

pred_precio = np.expm1(pred_log)

print("Precio estimado:", pred_precio[0])
```

In [36]:

```
resultados = pd.DataFrame({
    "Real": np.exp1(y_test),
    "Predicho": np.exp1(y_pred)
})

resultados.head(10)
```

In [37]:

```
import numpy as np

real = np.exp1(y_test)
pred = np.exp1(y_pred)

mape = np.mean(
    np.abs((real - pred) / real)
) * 100

print("MAPE =", mape)
```

In [38]:

```
### prueba con variables reducidas
X_sin_crypto = X.drop(
    columns=[
        col for col in X.columns
        if "cryptocurrency_" in col
    ]
)
```

```
]
)

print(X_sin_crypto.columns.tolist())
```

In [39]:

```
from sklearn.model_selection import train_test_split

X_train2, X_test2, y_train2, y_test2 = train_test_split(
    X_sin_crypto,
    y,
    test_size=0.2,
    random_state=2026
)
```

In [40]:

```
from sklearn.preprocessing import StandardScaler

scaler2 = StandardScaler()

X_train2_scaled = scaler2.fit_transform(X_train2)
X_test2_scaled = scaler2.transform(X_test2)
```

In [41]:

```
from sklearn.linear_model import LinearRegression

modelo2 = LinearRegression()

modelo2.fit(
    X_train2_scaled,
    y_train2
)
```

In [42]:

```
y_pred2 = modelo2.predict(
    X_test2_scaled
)
```

In [43]:

```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

mae2 = mean_absolute_error(
    y_test2,
    y_pred2
)

rmse2 = mean_squared_error(
    y_test2,
    y_pred2
) ** 0.5

r22 = r2_score(
    y_test2,
    y_pred2
)
```



```
print("MAE =", mae2)
print("RMSE =", rmse2)
print("R² =", r22)
```

In [44]:

```
import numpy as np

real2 = np.expm1(y_test2)
pred2 = np.expm1(y_pred2)

mape2 = np.mean(
    np.abs((real2 - pred2) / real2)
) * 100

print("MAPE =", mape2)
```

en esta prueba al eliminar el tipo de criptomoneda el rendimiento del modelo se distorción y dejó de ser confiable

El tipo de criptomoneda es el factor más importante para la predicción del precio actual. Al eliminar las variables categóricas asociadas a la criptomoneda, el coeficiente de determinación disminuyó de 0.9998 a 0.3791 y el error porcentual medio aumentó de 4.19% a 1239.55%, evidenciando una pérdida significativa de capacidad predictiva.

In [45]:

```
residuos = y_test - y_pred

plt.figure(figsize=(8,5))
plt.hist(residuos, bins=30)
plt.title("Distribución de residuos")
plt.xlabel("Residuo")
plt.ylabel("Frecuencia")
plt.show()
```

In [46]:

```
coeficientes.sort_values(
    "Abs",
    ascending=False
).head(10)
```

In [47]:

```
residuos = y_test - y_pred

print("Media residuos:", residuos.mean())
```

In [48]:

```
from statsmodels.stats.outliers_influence import variance_inflation_factor
import pandas as pd

vif = pd.DataFrame()

vif["Variable"] = X.columns

vif["VIF"] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])
]
```

```
vif = vif.sort_values(  
    by="VIF",  
    ascending=False  
)  
  
print(vif)
```